

GenAI. Autoregressive models. PixelCNN, PixelRNN

A. Ryzhikov¹, S. Ali¹, A. Akhmatova¹, T. Ramazyan¹, K. Aithadzhaev¹, A.
Khizhik¹

 ¹National Research University Higher School of Economics

March 28, 2025

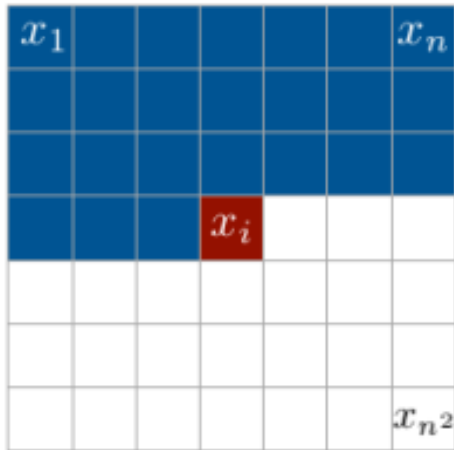


LAMBDA • HSE

PixelRNN



PixelRNN



PixelRNN

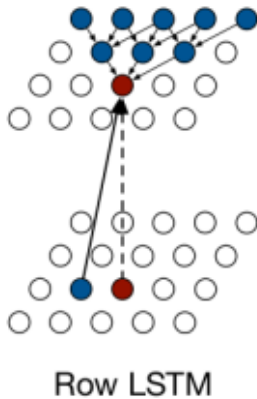
$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p(x_i \mid x_1, \dots, x_{i-1})$$

$$p(x_i, R \mid \mathbf{x}_{<i}) p(x_i, G \mid \mathbf{x}_{<i}, x_i, R) p(x_i, B \mid \mathbf{x}_{<i}, x_i, R, x_i, G)$$

RowLSTM



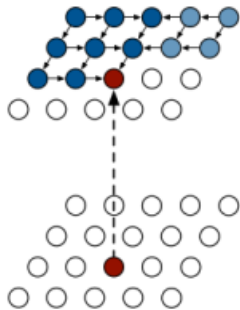
RowLSTM



Diagonal BiLSTM

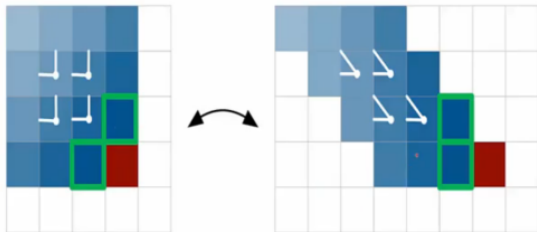


Diagonal BiLSTM



Diagonal BiLSTM

Diagonal BiLSTM



PixelCNN: Basic Model



PixelCNN: Overview

- ▶ Autoregressive image generation model.
- ▶ Uses Masked Convolutions.
- ▶ Each pixel value depends on previous pixels.
- ▶ Trained via Maximum Likelihood Estimation (MLE).

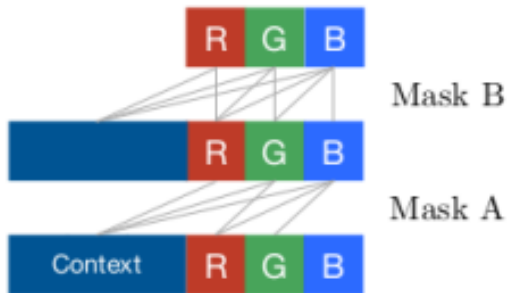
PixelCNN: Masked Convolutions

- ▶ Mask prevents access to future pixels.
- ▶ Example mask for 3x3 kernel:

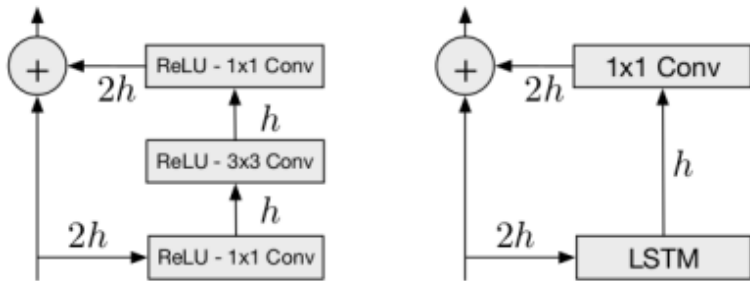
1	1	1	1	1
1	1	1	1	1
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0

PixelCNN: Masked Convolutions

$$p(x_i, R \mid \mathbf{x}_{<i})p(x_i, G \mid \mathbf{x}_{<i}, x_i, R)p(x_i, B \mid \mathbf{x}_{<i}, x_i, R, x_i, G)$$



PixelCNN: Residual Connections



Limitations of PixelCNN

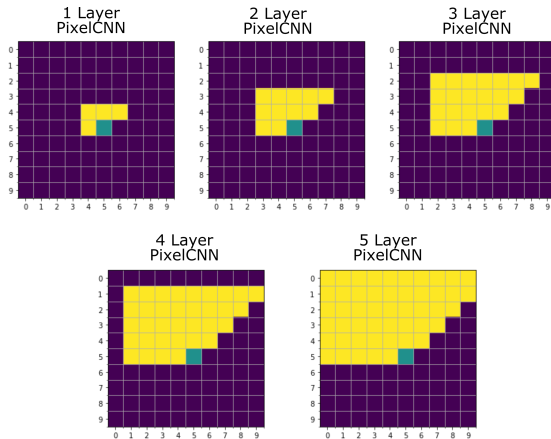


Limitations of Original PixelCNN

- ▶ Limited receptive field.
- ▶ Difficulty in modeling global dependencies.
- ▶ Blind zones when stacking masked layers.

Masked Convolutions: Blind Zone Problem

- ▶ Naive stacking limits receptive field.
- ▶ Example of the problem when stacking:



Gated PixelCNN



Gated PixelCNN: Improvements

- ▶ Introduces gated activations:

$$y = \tanh(W_f * x) \odot \sigma(W_g * x)$$

- ▶ Better local pixel dependencies.
- ▶ Higher quality images.
- ▶ Vertical and horizontal stack of convolutions.

Vertical and Horizontal Stacks

- ▶ **Vertical Stack:** Processes rows top-to-bottom.
- ▶ **Horizontal Stack:** Processes pixels left-to-right.
- ▶ Together they capture full autoregressive context.

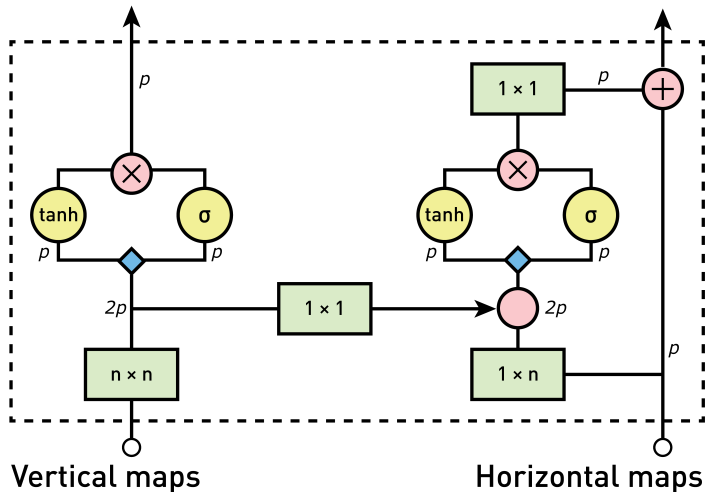
Why Split Vertical and Horizontal?

- ▶ **Vertical Stack:** Sees pixels above.
- ▶ **Horizontal Stack:** Sees pixels to the left.
- ▶ Vertical output is injected into horizontal via 1×1 conv.
- ▶ Prevents blind zones early in the network.

Interaction of Stacks

- ▶ Vertical stack: $n \times n$ masked conv $\rightarrow$ gated unit.
- ▶ Horizontal stack: $1 \times n$ masked conv $\rightarrow$ gated unit.
- ▶ Vertical influences horizontal via 1×1 conv.
- ▶ Residual connection added after gating.

Gated PixelCNN



PixelCNN++: Key Innovations



PixelCNN++: Key Innovations

- ▶ Mixture of Discretized Logistics.
- ▶ Joint RGB modeling.
- ▶ Down/Up-sampling architecture.
- ▶ Residual connections.

Mixture of Discretized Logistics

- ▶ Predict:
 - Means (μ_k)
 - Log-scales ($\log s_k$)
 - Mixture weights (π_k)
- ▶ Continuous modeling of pixel values.
- ▶ Better approximation of pixel distributions.

$$p(x) = \sum_{k=1}^K \pi_k \cdot \left[\sigma \left(\frac{x + 0.5 - \mu_k}{s_k} \right) - \sigma \left(\frac{x - 0.5 - \mu_k}{s_k} \right) \right]$$

- ▶ for *grayscale*, the model predicts $3K$ values for each pixel
- ▶ for *RGB*, the model predicts $10K$ values for each pixel

Joint RGB Modeling

- ▶ Predict R, then G (conditioned on R), then B (conditioned on R, G).
- ▶ Captures inter-channel dependencies.
- ▶ More realistic color generation.

Conditioning Coefficients for RGB Channels

- ▶ PixelCNN++ conditions RGB channels inside each pixel using linear dependencies:
 - Red is predicted directly.
 - Green depends on Red.
 - Blue depends on Red and Green.

Formulas (per mixture component k):

$$x_{i,R} \sim \text{Logistic}(\mu_{R,k}, s_{R,k})$$

$$x_{i,G} \sim \text{Logistic}(\mu_{G,k} + c_{G,k} \cdot x_{i,R}, s_{G,k})$$

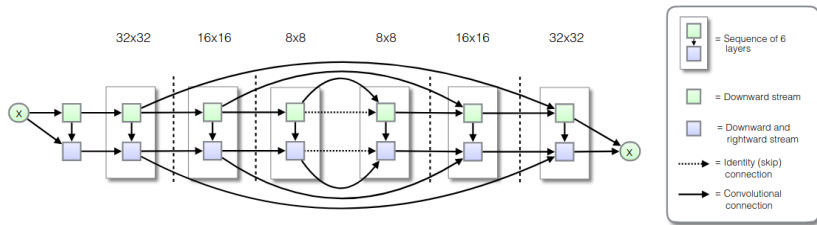
$$x_{i,B} \sim \text{Logistic}(\mu_{B,k} + c_{B1,k} \cdot x_{i,R} + c_{B2,k} \cdot x_{i,G}, s_{B,k})$$

Purpose of Coefficients:

- ▶ $c_{G,k}$: models dependence of Green on Red
- ▶ $c_{B1,k}, c_{B2,k}$: model dependence of Blue on Red and Green
- ▶ Enables efficient joint RGB modeling without full joint distribution

Downsampling/Upsampling Architecture

- ▶ Downsampling for global context.
- ▶ Upsampling to restore resolution.
- ▶ Residual connections to pass information.
- ▶ Only first block is masked.



Model Comparison Summary

Model	Architecture	Generation Speed	Image Quality
PixelCNN	Masked Convolutions	Slow	Medium
Gated PixelCNN	Masked + Gated Units	Slow	Higher
PixelCNN++	Deep Stack + Down/Up	Slow	Very High

Thank you for your attention!



Artem Ryzhikov aryzhikov@hse.ru